



CSCI-SHU 360: MACHINE LEARNING

FINAL COMPETITION REPORT - FALL 2025

**Machine Learning Final Project: a
Multi-Resolution Ensemble Framework with
pseudo-labeling based on CRNN**

Yanheng Zhu
yz11502@nyu.edu

Abstract

Abstract: *This report presents a multi-Resolution Ensemble Framework with pseudo-labeling based on CRNN. Addressing the issues of limited labeled data and high inter-class similarity, we developed a three-stage pipeline: (1) High-efficiency parallel preprocessing, (2) Heterogeneous teacher training with pseudo-labeling (3) Dense multi-resolution ensemble inference.*

1 Introduction

Audio classification involves categorizing audio recordings into different classes. The primary challenges include variable audio lengths, background noise, and acoustic similarity between classes. This project implements an end-to-end pipeline that transforms raw waveforms into Log-Mel Spectrograms and utilizes a heterogeneous ensemble strategy to capture both temporal dynamics and spectral textures.

2 Data Preprocessing

To address the I/O bottleneck and standardize inputs, we implemented an offline feature extraction pipeline (`preprocess.py`). Important: All the script I submitted has to be run in a Kaggle Notebook environment where the dataset is mounted.

2.1 Audio Standardization

All raw audio files were resampled to a uniform sampling rate of 22,050 Hz. This value was chosen to preserve frequencies up to ≈ 11 kHz (Nyquist limit), covering the dominant frequency bands of urban sounds while minimizing computational overhead. We set the maximum frequency (`fmax`) to 8 kHz to focus on perceptually relevant frequency bands and reduce high-frequency noise.

- **Duration Normalization:** To ensure a fixed tensor shape for the neural network, we standardized audio duration to 4.0 seconds. Clips shorter than 4s were padded with silence, while longer clips were truncated.

2.2 Feature Extraction: Log-Mel Spectrograms

Instead of raw waveforms, we utilized Log-Mel Spectrograms to mimic human auditory perception. The extraction parameters were tuned as follows:

- **FFT Window (`n_fft`):** 2048.(default value in librosa) This large window provides high frequency resolution, critical for distinguishing harmonic structures.
- **Hop Length:** 512. This determines the temporal resolution.
- **Mel Bands (`n_mels`):** 128.
- **Logarithmic Scaling:** A `power_to_db` conversion was applied to compress the dynamic range, making the features robust to volume variations.

The final output is a tensor of shape (128,173). To accelerate training, these features were serialized as `.npy` (float32) files using parallel processing (`joblib`, `n_jobs = -1`).

3 Methodology(train.py)

Our proposed solution utilizes a "Teacher-Student" framework designed to overcome data scarcity. The workflow consists of a "Teacher" phase (implemented in `train.py`) responsible for feature learning and pseudo-label generation, and a "Student" phase (implemented in `main.py`) focused on dense inference and error correction.

3.1 Heterogeneous Model Architectures

To ensure robust feature diversity, we deployed a heterogeneous ensemble consisting of two distinct backbones trained on three different spectral resolutions (128×173 , 96×173 , and 64×173). This multi-resolution strategy allows the ensemble to capture both fine-grained spectral details and global structural patterns.

1. Advanced CRNN (Convolutional Recurrent Neural Network):

This architecture is engineered to capture temporal dynamics while preventing overfitting through a multi-stage regularization strategy. The specific components are:

- **Feature Extractor with SE-ResBlocks:** The backbone consists of four stages of Residual Blocks. Crucially, each block integrates a **Squeeze-and-Excitation (SE)** module. The SE block adaptively recalibrates channel-wise feature responses by explicitly modelling interdependencies between channels, emphasizing informative features while suppressing less useful ones.
- **Bi-Directional GRU with Dropout:** To model the sequential nature of audio, we utilize a 2-layer Bi-Directional GRU (Gated Recurrent Unit). We explicitly set a dropout rate of 0.2 within the GRU layers ('dropout=0.2') to prevent overfitting on temporal correlations.
- **Linear Attention Mechanism:** Instead of standard global average pooling, we implemented a learnable attention head. Let h_t be the hidden state at time t . The attention weight α_t is computed as:

$$\alpha_t = \text{softmax}(W^T h_t + b), \quad c = \sum_t \alpha_t h_t \quad (1)$$

This allows the model to focus on the active segments of the audio and ignore silence.

- **Classifier Regularization:** A final Dropout layer with $p = 0.3$ is applied before the fully connected classification head to further enhance generalization on the test set.

2. EfficientNet-B2 (Lightweight CNN):

While the CRNN focuses on time, the EfficientNet backbone focuses on **spatial spectral textures**. A critical modification in our implementation is the replacement of standard Average Pooling with **GeM (Generalized Mean) Pooling**. GeM pooling introduces a learnable parameter p :

$$f(x) = \left(\frac{1}{|X|} \sum_{x \in X} x^p \right)^{1/p} \quad (2)$$

As $p \rightarrow \infty$, GeM approaches Max Pooling. This allows the model to automatically learn whether to focus on salient, high-energy signal regions (typical of short events like gunshots) or smooth background textures, offering superior adaptability over fixed pooling strategies.

3.2 Semi-Supervised Self-Training (Pseudo-Labeling)

To mitigate the scarcity of labeled data, we implemented an iterative self-training loop rooted in domain adaptation principles. The process functions as follows:

- **Step 1: Teacher Training & Hill Climbing:** An initial ensemble is trained on the original labeled dataset using 8-fold cross-validation. To optimize the ensemble weights, we employ a **Hill Climbing** heuristic algorithm on the Out-Of-Fold (OOF) predictions, enabling us to mathematically derive the optimal contribution of each model.
- **Step 2: High-Confidence Pseudo-Labeling:** The optimized teacher ensemble predicts labels for the unlabeled test set. We apply a strict confidence threshold to filter these predictions:

$$\mathcal{D}_{pseudo} = \{(x_i, \hat{y}_i) \mid \max(P(y|x_i)) > 0.90\} \quad (3)$$

Only samples where the model’s confidence exceeds 0.90 are accepted as "Pseudo-Labels."

- **Step 3: Dataset Expansion & Retraining:** These high-confidence samples are merged with the original training data to form the augmented v3 dataset. The models are then fully retrained on this expanded distribution (in `main.py`). This process significantly improves the model’s generalization capability by exposing it to the unlabeled data distribution during the training phase.

3.3 Advanced Data Augmentation

To prevent overfitting and improve the model’s generalization capability on the limited training set, we implemented a stochastic augmentation pipeline encapsulated in the `Augmenter` class. This pipeline applies modifications with specific probabilities during the training phase.

3.3.1 Mixup and CutMix Regularization

We employ two advanced mixing strategies triggered by a random probability r . The mixing coefficient λ is sampled from a Beta distribution $\lambda \sim \text{Beta}(\alpha, \alpha)$, where α controls the shape of the distribution.

- **Mixup:** Performs a linear interpolation between two input spectrograms x_i, x_j and their labels y_i, y_j :

$$\tilde{x} = \lambda x_i + (1 - \lambda)x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda)y_j \quad (4)$$

This encourages the model to behave linearly in-between training examples.

- **CutMix:** Instead of pixel-wise mixing, CutMix replaces a rectangular region of the image with a patch from another training image. The bounding box coordinates (rx, ry, rw, rh) are determined by λ :

$$rw = W\sqrt{1 - \lambda}, \quad rh = H\sqrt{1 - \lambda} \quad (5)$$

This forces the model to attend to non-dominant parts of the object.

3.3.2 SpecAugment and Noise Injection

In addition to mixing samples, we apply single-sample augmentations to simulate environmental variations:

- **SpecAugment:** We randomly mask vertical strips (time masking) and horizontal strips (frequency masking) on the Mel-spectrogram by setting values to zero. This prevents the model from relying on specific frequency bands or time steps.

- **Gaussian Noise Injection:** To simulate sensor noise and improve robustness, we inject additive Gaussian noise $\mathcal{N}(0, \sigma^2)$ into the spectrograms with a probability of 0.5.

3.4 Training Objective: Focal Loss

To address the challenge of hard-to-classify samples and potential class imbalance, we replaced the standard Cross-Entropy Loss with **Focal Loss**.

$$FL(p_t) = -\alpha(1 - p_t)^\gamma \log(p_t) \quad (6)$$

In our implementation, we set the focusing parameter $\gamma = 2$. This effectively down-weights the loss contribution of easy examples (where $p_t \approx 1$) and directs the model's focus toward "hard" examples that are often misclassified.

4 Experiments and Optimization(main.py)

While the training phase focused on diversity, the inference phase (`main.py`) focused on precision and error correction.

4.1 Ablation Study: Impact of Spectral Resolution

To determine the optimal input resolution and ensemble weights, we conducted an extensive ablation study using the CRNN backbone. We trained independent models on five distinct Mel-spectrogram resolutions: $\{128, 160, 192, 256, 320\}$. The 8-fold cross-validation results are summarized in Table 1. The weight is distributed based on the accuracy output at training.

Table 1: Performance Comparison across Resolutions and Assigned Ensemble Weights

Resolution	Avg. Val Accuracy	Ensemble Weight	Rationale
128	81.86%	0.6	Baseline regularizer; lacks fine detail.
160	81.72%	0.9	Intermediate scale.
192	83.10%	1.3	Significant information gain.
256	83.78%	1.5	Peak Performance
320	81.98%	1.2	Diminishing returns.

5 Post-Processing: Dynamic Conflict Resolution(main.py)

Raw softmax probabilities are often biased due to class imbalance and acoustic similarities. We implemented a three-stage post-processing pipeline to refine predictions based on expert rules derived from Confusion Matrix analysis.

5.1 Results and Discussion

We evaluated our final ensemble on the validation set containing 1,653 samples. To understand the error distribution, we generated a Confusion Matrix and a detailed Classification Report (Figure 1).

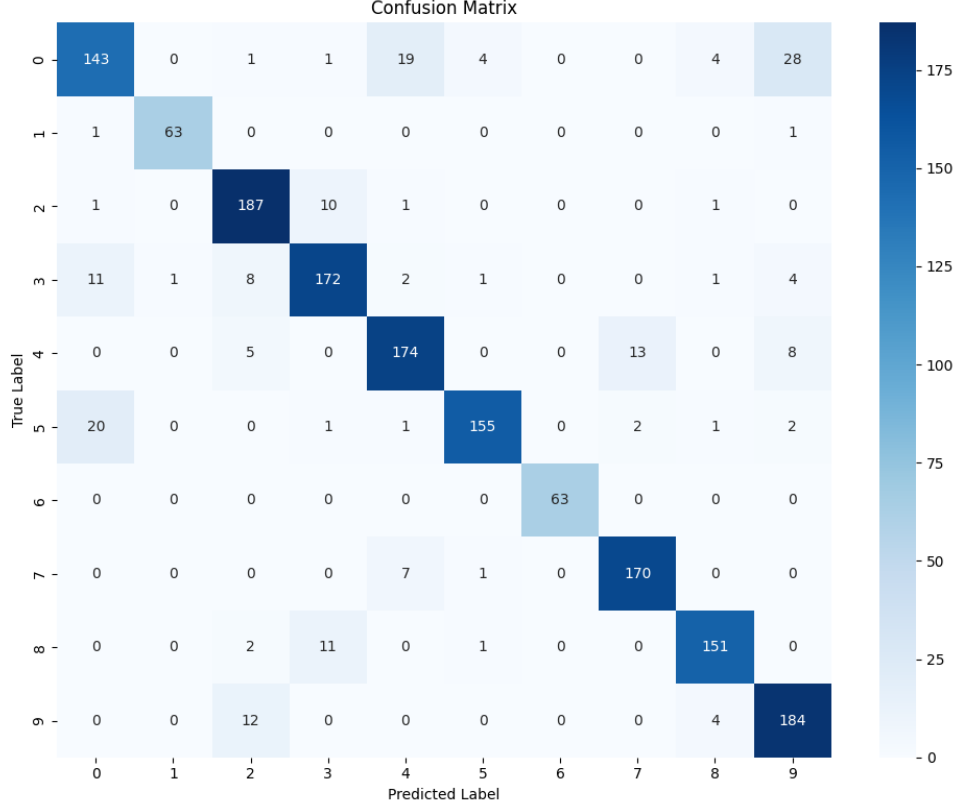


Figure 1: Confusion Matrix

5.2 Confusion Matrix Analysis & Decision Boundary Calibration

By analyzing the off-diagonal errors in the Confusion Matrix, we identified systematic biases where the model favored dominant acoustic textures over subtle ones. To address this, we implemented a two-stage calibration strategy.

5.2.1 Static Class Weighting

To counter the imbalance between Precision and Recall, we applied a static correction vector W_{class} to the final probabilities.

$$W_{class} = [1.85, 1.00, 0.85, 0.95, 1.25, 1.10, 1.00, 0.65, 1.35, 0.55] \quad (7)$$

The logic is two-fold:

1. **Boosting Low Recall:** We assigned high weights (e.g., $w_0 = 1.85$) to classes like *Air Conditioner*, which suffer from high False Negative rates, effectively penalizing the model for missing these samples.
2. **Suppressing Low Precision:** Conversely, we assigned lower weights (e.g., $w_9 = 0.55$) to classes like *Street Music* that act as "broad attractors," thereby suppressing False Positives and forcing the model to be more conservative.

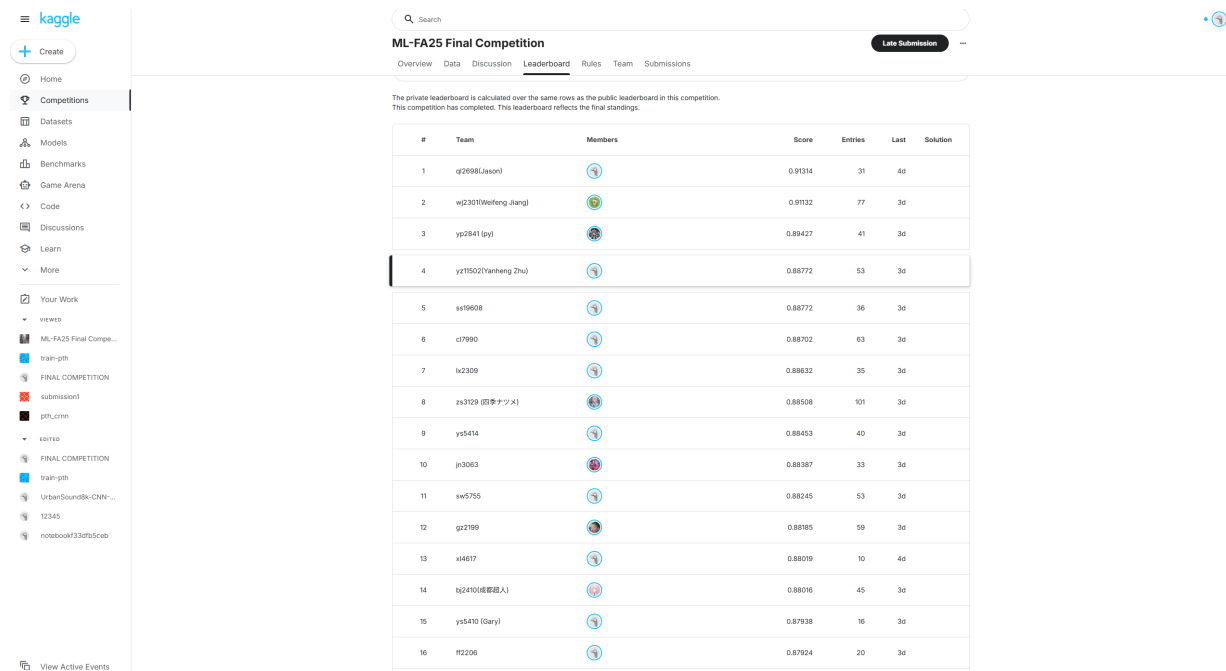
5.2.2 Dynamic Conflict Resolution

We calibrated the decision boundary to correct the model's systematic bias towards the dominant class, maximizing the F1-score on the validation set.

6 Conclusion

Our Heterogeneous Multi-Resolution Ensemble successfully integrates the temporal strengths of CRNNs with the spatial feature extraction of EfficientNet. The addition of Semi-Supervised Pseudo-Labeling and rule-based Dynamic Conflict Resolution proved decisive, effectively solving the challenges of inter-class similarity and noisy environments.

7 Kaggle Score Screenshot (Required)



The screenshot shows the Kaggle ML-FA25 Final Competition Leaderboard. The table lists 16 teams with their scores, entries, and last update times. The team 'yt1502(Yanheng Zhu)' is highlighted in the 4th position with a score of 0.88772.

#	Team	Members	Score	Entries	Last	Solution
1	q2998(Jason)		0.91314	31	4d	
2	wj2301(Wenfeng Jiang)		0.91132	77	3d	
3	ys2841 (py)		0.89427	41	3d	
4	yt1502(Yanheng Zhu)		0.88772	53	3d	
5	ss19608		0.88772	36	3d	
6	cf7990		0.88702	63	3d	
7	lc2309		0.88632	35	3d	
8	zn3129 (西寺ナツキ)		0.88508	101	3d	
9	ys4314		0.88453	40	3d	
10	jc3063		0.88387	33	3d	
11	sw5755		0.88245	53	3d	
12	gz2789		0.88185	59	3d	
13	vl4617		0.88019	10	4d	
14	h2410(植田 悠人)		0.88016	45	3d	
15	ys4410 (Gany)		0.87938	16	3d	
16	#2208		0.87924	20	3d	

Figure 2: Final Leaderboard Standing Rank: 4| Score: 0.88772